

Starting Serverless Workflows Faster: Planning CPU and Snapshots Along the Workflow

MSc thesis progress report 2 · Amirkabir University of Technology · September 2026

Title: Improving the function runtime environment in FaaS to reduce workflow cold start

This report answers the review of report 1. All results are from simulation; the measurements that confirm the model on our test machine are planned (section 7).

1. The review, and what changed

your point	what was wrong	what we did
Not every function should be restored. Some are unsuitable, inefficient or too costly to snapshot.	Report 1 said "restore every stage".	Each stage now gets its own start mode: warm, snapshot restore, or cold start with extra CPU. A snapshot is built only where it is safe and affordable. A stage without one still starts fast: with extra CPU it beats SnapStart-style restores (section 5).
No creativity: "you just restore earlier".	The timing idea is Xanadu's (2020). Applying it to snapshots was a small step.	A new mechanism built on our own measurement, the vCPU cliff: the platform plans the CPU given to sandboxes <i>while they start</i> , using the workflow graph (sections 2–4).
Nothing optimised, no baseline improved.	We compared only against our own system without our change.	The problem now has an objective, an exact algorithm and a fast online rule. It is compared against published and industrial baselines, including Google Cloud Run's startup CPU boost (section 5).

2. The observation: start-up is starved of CPU

Starting a function is CPU work: restoring a snapshot, loading classes, JIT compilation. Serverless functions get small CPU shares (AWS Lambda gives a 512 MB function about 0.3 vCPU). Our measurements show that start-up suffers most from this:

- JVM warm-up is **13–35× slower** at 0.25 vCPU than at 4 vCPU (exp14).
- The same warm-up takes **3.59 s at 0.25 vCPU and 0.86 s at 1 vCPU**: 4.2× faster, for **the same CPU-seconds** (0.90 vs 0.86; exp-b).
- Spring Boot cold starts take 2.3–26.8 s and snapshot restores 0.64–9.1 s depending on the CPU quota (exp12).

So giving a starting sandbox more CPU does the same work sooner, without using more CPU in total. A server's CPU is mostly idle, but it is limited, and when a workflow starts cold, several sandboxes start at once and compete for it. **Who gets the spare CPU, and when, decides how fast the workflow starts.**

3. The idea

When a workflow arrives and some of its stages have no warm sandbox, the platform decides for each such stage:

- how it starts**: from a snapshot, or a cold start, or it waits for a warm sandbox;

2. **when it starts:** ahead of need, from the workflow graph;
3. **how much CPU it gets while starting**, from the server's spare CPU. After start-up it goes back to its normal share.

The workflow graph gives the key information: which start-ups are on the critical path, and how much slack the others have.

4. The algorithm

Step 1: deadlines from the graph. A result from report 1 (Theorem 1) gives, for a latency target L , the time $D_v = L - \ell(v)$ by which each stage must be ready. Here $\ell(v)$ is the work still ahead of stage v in the workflow. So every start-up has its own deadline.

Step 2: the CPU plan. Each start-up needs a known amount of CPU work and can use at most about one core; the server has limited spare CPU. The question "can every start-up meet its deadline?" is then a **maximum-flow problem** (Horn, 1974). A binary search on L gives the **exact optimum** and the CPU share of every start-up over time. This is a classical problem, project scheduling in which activities run faster with more resource. Report 1's timing results are its special case with fixed CPU.

For use at run time, a fast **online rule** recomputes the shares whenever a start-up begins or ends: - **one workflow starting:** balanced shares, so that all its start-ups are equally late. This is within 0–9% of the exact optimum on our test cases. - **several workflows starting at once:** the most urgent start-up first (earliest deadline). - **stages that may not run** (an if/else not yet decided) get only CPU that nothing certain can use. With at most one spare core they are not started early at all: without parallelism, starting them early only takes CPU from stages that will run. - **just in time:** each start-up waits, holding no memory, until its latest start from the exact plan, and then runs at full speed. It gives up no reachable deadline. With one spare core it reaches the best latency and the least memory together, as report 1's just-in-time restore did for fixed restore times.

Step 3: which stages get a snapshot. A snapshot is built at deployment only for functions where it is safe (random-number state and secrets can be reset) and affordable. At run time, each stage uses the cheapest start that meets its deadline.

5. Results (simulation)

Simulator: our discrete-event simulator of serverless workflows, extended with CPU as a shared server resource. Functions run at 0.25 vCPU. Restore and cold-start costs are the thesis's measured values. With the CPU model switched off, the simulator reproduces all earlier results exactly.

Baselines: - **SnapStart-style:** restore each stage's snapshot when its request arrives, at the function's normal CPU share. - **Cloud Run-style:** the same, plus Google's startup CPU boost: every starting sandbox gets an equal share of the spare CPU. - **Look-ahead (report 1):** restore ahead along the workflow, at the normal CPU share. - **Look-ahead + equal boost:** our timing with the industrial boost.

One cold workflow (functions at 0.25 vCPU; a server with 2 cores free for this workflow; mean end-to-end latency in seconds, 50 runs with timing noise):

workflow	SnapStart-style	Cloud Run-style	look-ahead (report 1)	look-ahead + equal boost	CPU plan
3-stage chain	8.64	2.16	3.63	1.20	1.14
8-stage chain	23.47	5.88	5.20	3.20	3.05
fan-out / fan-in	7.04	2.43	3.75	2.01	1.97
ML pipeline	8.47	2.12	3.45	1.17	1.13
if/else router	12.28	3.08	4.61	2.79	2.06
trip booking (saga)	12.41	3.11	4.28	2.63	1.64

- **Against Cloud Run's startup boost**, over eight workflow shapes, the CPU plan is:
 - **19–48% faster** with 2 free cores;
 - **41–71% faster** with 4 free cores;
 - equal with 1 free core. With one core, start-ups cannot run in parallel, and the plan correctly adds nothing.
- **Against look-ahead with an equal boost**, it is 0–37% faster. The largest gains are on workflows with if/else branches, because it does not spend scarce CPU on branches that may not run.
- **Against SnapStart-style restore at the function's normal CPU share**, it is 3.6–14× faster with 2–4 free cores.
- **Against the exact optimum** (fluid model), the online rule is within 0–9%.
- **Just in time** (section 4) changes these latencies by –8% to +1.3% (if/else router: 2.06 → 1.89 s) and cuts memory (section 7).

A burst: 16 cold workflows of these shapes arrive within one second (mean / 99th percentile, seconds):

free cores	SnapStart-style	Cloud Run-style	look-ahead (report 1)	look-ahead + equal boost	CPU plan
2	21.2 / 34.7	19.9 / 26.0	28.9 / 31.2	28.7 / 30.4	16.7 / 25.5
4	12.8 / 25.8	9.9 / 14.1	14.5 / 16.3	14.1 / 15.2	8.1 / 12.6
8	11.6 / 24.6	5.1 / 8.4	7.5 / 9.1	6.9 / 7.6	3.9 / 6.4

Report 1's look-ahead is *slower* than restoring on demand in a burst: all restores start at once and compete for the CPU. The CPU plan fixes this. Its mean is 16–24% below the Cloud Run-style baseline, and its 99th percentile 2–24% below. Just in time, it also holds **10–30% less memory than the Cloud Run-style baseline** (2 cores: 142 vs 204 GB·s per burst; 8 cores: 45 vs 50), and 76–82% less than without it.

Without any snapshot: cold prewarm along the workflow with the CPU plan beats SnapStart-style snapshot restores for 7 of 8 workflow shapes on 2 free cores (3-stage chain: 4.65 s vs 8.64 s), and for all 8 on 4. So a function whose snapshot is unsafe or not worth storing still starts fast.

Real traffic: the Azure Functions 2021 trace (68 workflows, 433,000 calls over 3 days; 32 GB memory, 64 cores; mean / 99th percentile, seconds). "Cold workflows" are the 794 calls to a workflow idle for more than 10 minutes.

	SnapStart-style	Cloud Run-style	look-ahead (report 1)	look-ahead + equal boost	CPU plan
cold workflows	9.36 / 24.8	2.35 / 6.20	3.41 / 6.28	0.84 / 1.53	0.84 / 1.53
all calls	1.49 / 14.3	0.87 / 3.52	1.31 / 9.24	0.80 / 2.27	0.80 / 2.27
CPU-hours	44.7	37.2	53.5	42.5	42.5

(SnapStart-style uses more CPU than Cloud Run-style because its sandboxes start slowly, so more of them are started to serve the same calls.)

- **Cold workflows** are 2.8× faster on average and 4× faster at the 99th percentile than with Cloud Run's boost. The 99th percentile over all calls is 36% lower.
- **The cost** is 14% more CPU-hours, because more sandboxes are started ahead.
- **The CPU plan gives the same result as an equal split here.** On a server sized for this traffic (32–64 cores), spare CPU is almost always available, so how it is shared rarely matters. The plan pays off where CPU is contended: small servers and bursts (above). On real traffic the gain comes from combining look-ahead with extra CPU while starting.
- **Report 1's look-ahead without extra CPU is poor at small CPU shares:** its parallel restores starve each other.

6. What is new

- **Snapshot systems** (SnapStart, Pronghorn, REAP) make one restore faster and start it when the request arrives.
- **Workflow-aware systems** (Xanadu, ORION) time the start of cold containers along the graph. ORION and Aquatope choose one CPU size per stage for its whole run.
- **Cloud Run's startup CPU boost** gives every starting instance the same extra CPU. It knows nothing about workflows or competing start-ups.
- **Ours:** start-ups timed along the workflow *and* given extra CPU while they start, with the CPU planned per start-up and over time from the workflow's deadlines. There is an exact algorithm (max-flow), a fast online rule, and the choice of start mode per stage. It rests on our own measurement that start-up is where small CPU shares hurt most.
 - On real traffic with a well-sized server, most of the gain comes from combining look-ahead with extra start-up CPU.
 - The planning itself adds most where CPU is contended.

7. Honest limits, and next steps

- **The CPU speed-up is modelled, not yet measured.** The model assumes a start-up runs proportionally faster with CPU, up to one core, at the same CPU-seconds. That is what we measured for JVM warm-up (exp-b). For snapshot restores and cold starts it is the first thing to measure (next step 1).
- **It does not save snapshots on these workflows.** With short stages (about 50 ms), every stage still needs its snapshot under the CPU plan: a cold start is about 4× the CPU work of a restore. Snapshots can be dropped where later stages have enough slack, which in our model needs stages of about a second. Report 1's cost analysis still applies: in money, most snapshots do not pay for themselves.
- **The burst rule depends on the mix.** "Most urgent first" is best for mixed bursts. For bursts of identical workflows, the balanced rule was better in the fluid model.

- **Memory.** Starting every restore when the workflow arrives made sandboxes wait: up to $11\times$ the memory-time of the Cloud Run-style baseline. Holding each start-up back to its latest start (just in time) fixes most of it at the same latency:
 - one workflow: 49–88% less on one spare core, where chains then hold exactly the baseline's memory-time, and up to 73% less on two;
 - a burst: 76–82% less, below the baseline;
 - plenty of spare CPU (four idle cores for one workflow): up to 33% less. There the plan still holds up to $2.6\times$ the baseline's memory-time.
- **It needs spare CPU, and contention.** With one free core or less there is nothing to plan: the result equals the Cloud Run-style boost. With plenty of spare CPU, as on the Azure trace with a 32–64 core server, an equal split does as well. The CPU plan's own gains appear in between: small servers and bursts.

Next steps:

1. **Measure on the test machine** (1–2 weeks). How a CRaC restore and a JVM cold start speed up with CPU, what that costs in CPU-seconds, and whether the CPU limit can be raised only during start-up (cgroup `cpu.max`). Then rerun the simulations with the measured curve.
2. **Implement in Apache OpenWhisk** (4–6 weeks). An orchestrator that sets each starting container's CPU limit from the plan, plus snapshot restore for the stages that use it.
3. **Evaluate** against the baselines above, on benchmark workflows (SeBS-Flow) and trace replay.
4. **Write.**